# AI StudyBuddy API

AI StudyBuddy (also referred to as LearnMate) is a REST API for helping students turn study material into revision resources. It uses Node.js, Express, MongoDB/Mongoose, JWT authentication, and Google Gemini AI.

> ***Documentation basis:*** *This guide is based on the supplied project brief. The source code and a running API were not available for inspection, so endpoint payloads, response schemas, exact route behavior, and implemented features must be checked against the actual project before treating them as verified.*

## Project links

- Backend demo video
- Demo video
- Source code (Google Drive folder)

## Contents


- Overview
- Features and roles
- Technology
- Architecture
- Data model
- Requirements
- Setup
- Configuration
- Run the server
- API reference
- Authentication and security
- Typical workflow
- Troubleshooting
- Known gaps in the source brief


## Overview

The API provides authenticated students with a place to manage study material and request AI-generated summaries, flashcards, quizzes, and study plans. MongoDB stores accounts, materials, and generated resources. The server calls Google's Gemini API to produce learning content.

## Features and roles

### Student

- Register and sign in.
- Upload or provide study material.
- Generate a summary, flashcards, a multiple-choice quiz, or a study plan.
- Retrieve and manage saved materials and generated resources, where supported by the implementation.

### Administrator

The brief describes administrator permissions for managing users, monitoring service usage, and maintaining platform integrity. The exact administrative routes and authorization rules are not listed.

## Technology

| Component | Purpose |
|---|---|
| Node.js | JavaScript server runtime; the brief specifies v16 or later. |
| Express.js | HTTP server and REST route handling. |
| MongoDB | Document database. |
| Mongoose | MongoDB schemas, validation, and data access. |
| JSON Web Token (JWT) | Authentication for protected routes. |
| bcryptjs | Password hashing. |
| Google Gemini API | AI content generation; the brief names Gemini 2.5 Flash. |
| Multer | File upload handling. |
| dotenv | Loads local environment configuration. |
| Postman or Thunder Client | Manual API requests. |

The listed dependencies also include `cors` and `@google/genai`.

## Architecture

The brief describes a modular MVC-style backend:

1. **Express server** loads configuration, connects to MongoDB, installs middleware, registers routes, and listens for requests.
2. **Routes and controllers** validate and process requests, invoke models or AI services, and return JSON.
3. **Middleware** verifies JWTs, applies role checks, and handles uploaded files.
4. **Models** represent users, materials, and generated learning resources through Mongoose.
5. **Gemini service** sends prompts to Google's Gemini API and returns generated study content.

The frontend is described as a React client, but it is outside the scope of this API guide.

## Data model

The brief proposes these entities and relationships. Confirm the actual schemas in the repository.

| Entity | Main fields described | Relationship |
|---|---|---|
| User | `_id`, `name`, `email`, hashed `password`, `role`, `createdAt` | Owns materials, learning resources, and plans. |
| Study Material | `_id`, `userId`, `title`, `subject`, `content`, `createdAt` | Belongs to a user; can be used to generate resources. |
| Summary | `_id`, `userId`, `materialId`, `summary`, `createdAt` | Associated with its owner and source material. |
| Flashcard | `_id`, `userId`, `materialId`, `question`, `answer`, `createdAt` | Multiple cards may be generated from a material. |

| Entity | Main fields described | Relationship |
| --- | --- | --- |
| Quiz | `_id`, `userId`, `materialId`, `questions`, `createdAt` | Contains questions generated from a material. |
| Study Plan | `_id`, `userId`, `studyPlan`, `examDate`, `createdAt` | A user may create multiple plans. |

# Requirements

- Node.js v16 or later and npm v8 or later (as listed in the brief).
- MongoDB instance and a connection string.
- Google Gemini API key.
- Postman or Thunder Client for manual API exploration.

# Setup

Open the project directory in a terminal and install dependencies:

```
npm install
```

The project brief lists these packages as the intended dependency set:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
npm install --save-dev nodemon
```

The actual `package.json` is authoritative; avoid reinstalling or changing versions if the repository already defines them.

# Configuration

Create a `.env` file in the project root, or use the configuration method already present in the repository:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=replace_with_a_long_random_secret
GEMINI_API_KEY=your_gemini_api_key
```

Keep `.env` private and out of version control. Never commit real database credentials, JWT secrets, or AI provider keys. The names above come from the supplied brief; check the source for the exact names it reads.

# Run the server

The brief says to start the development server with:

```
npm start
```

The actual command depends on the `scripts` configured in `package.json`. If `npm start` is undefined, inspect the available scripts with `npm run` and use the project's configured development command. The brief expects the server to connect to MongoDB before serving requests and gives port `5000` as an example.

# API reference

The paths below are the routes stated in the project brief. HTTP methods are provided only where the brief specifies them. Request body shapes, response examples, validation rules, and error codes were not provided and should be documented from the implementation or an API collection.

| Method | Path | Authentication | Description |
| --- | --- | --- | --- |
| `POST` | `/api/auth/register` | Public | Register a user; password is described as hashed before storage. |

| Method | Path | Authentication | Description |
|---|---|---|---|
| POST | /api/auth/login | Public | Authenticate a user and issue a JWT. |
| POST | /api/material/upload | JWT required (intended) | Upload study material. The brief describes Multer-based file handling. |
| POST | /api/materials/:id/summarize | JWT required (intended) | Generate a concise summary from a material. |
| POST | /api/ai/flashcards | JWT required (intended) | Generate flashcards from study content. |
| POST | /api/ai/quiz | JWT required (intended) | Generate a multiple-choice quiz. |
| POST | /api/ai/study-plan | JWT required (intended) | Generate a personalized study plan. |

### Authentication header

For protected routes, the intended convention is a bearer token:

```
Authorization: Bearer <JWT>
```

Verify the token format and exact middleware behavior in the project code. The brief does not list material read/update/delete endpoints, saved-resource retrieval endpoints, or administrator route paths.

### Request and response details

The project brief does not define JSON or multipart request fields, response examples, status codes, pagination, or error formats. Use the route handlers and validation middleware as the source of truth. For file uploads, confirm the expected multipart field name, accepted formats, size limits, and whether files or extracted text are stored.

## Authentication and security

- Hash passwords with bcryptjs before database storage.
- Sign and verify JWTs with a secret provided through environment configuration.
- Protect user and AI routes with authentication middleware.
- Apply role-based checks to administrator-only functions.
- Validate and limit uploaded files and request sizes.
- Keep API keys and database credentials out of source control and logs.
- Ensure users can only read or modify their own materials and generated resources.

These are project design requirements described in the brief; verify their implementation in the repository.

## Typical workflow

1. Register with `POST /api/auth/register`.
2. Log in with `POST /api/auth/login` and retain the returned token securely.
3. Send the token as a bearer token when calling protected endpoints.
4. Upload or create study material.
5. Request a summary, flashcards, quiz, or study plan.
6. Review and save the generated result if the relevant persistence routes are implemented.

# Troubleshooting

| Symptom | Checks |
| --- | --- |
| Server fails to start | Confirm Node.js is installed, dependencies are installed, required environment variables exist, and the configured port is available. |
| MongoDB connection fails | Check `MONGO_URI`, database availability, network access, and MongoDB credentials. |
| Protected route returns an authorization error | Check that login succeeded, the token is current, and the `Authorization` header uses the expected format. |
| AI generation fails | Check `GEMINI_API_KEY`, provider access/quota, network connectivity, and server logs. Do not expose the key in logs or client responses. |
| Upload is rejected | Check the route's expected form field, supported file types, size limit, and Multer configuration. |
| `npm start` is unavailable | Review the `scripts` section of `package.json` and use a script that the project actually defines. |

# Known gaps in the source brief

This guide documents the supplied project brief, not a code audit. The brief does not provide the repository's actual folder tree, exact request/response schemas, all HTTP methods, status/error conventions, upload limits, or a Postman collection. It also mentions database entities beyond the two collections enumerated later in the brief. Confirm these details against the source code before publishing this as a contract for API clients.